

K8s Demonstration - Resume builder

Monolithic Architecture (Kubernetes-based)

1. Start Minikube:
`minikube start --vm-driver=docker`
2. Set up Minikube's Docker environment:
`eval $(minikube docker-env)`
3. Build the Docker image for the monolithic application:
`docker build -t resume-builder .`
4. Create and apply the Kubernetes YAML file (resume-builder.yaml):
`kubectl apply -f resume-builder.yaml`
5. Verify the deployment:
`kubectl get pods`

```
service/resume-builder created
(dev) → Resume_Builder git:(main) ✕ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
resume-builder-5d69fcddcc-mm9fk     1/1     Running   0           6s
```

6. Access the application:
`minikube service resume-builder`
7. View all resources:
`kubectl get all`

```
(dev) → Resume_Builder git:(main) ✕ kubectl get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/resume-builder-5d69fcddcc-mm9fk 1/1     Running   0           81s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
service/kubernetes                  ClusterIP     10.96.0.1     <none>       443/TCP          38m
service/resume-builder              LoadBalancer 10.101.220.152 <pending>    5002:30001/TCP   81s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/resume-builder      1/1     1             1           81s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/resume-builder-5d69fcddcc 1         1         1       81s
(dev) → Resume_Builder git:(main) ✕
```

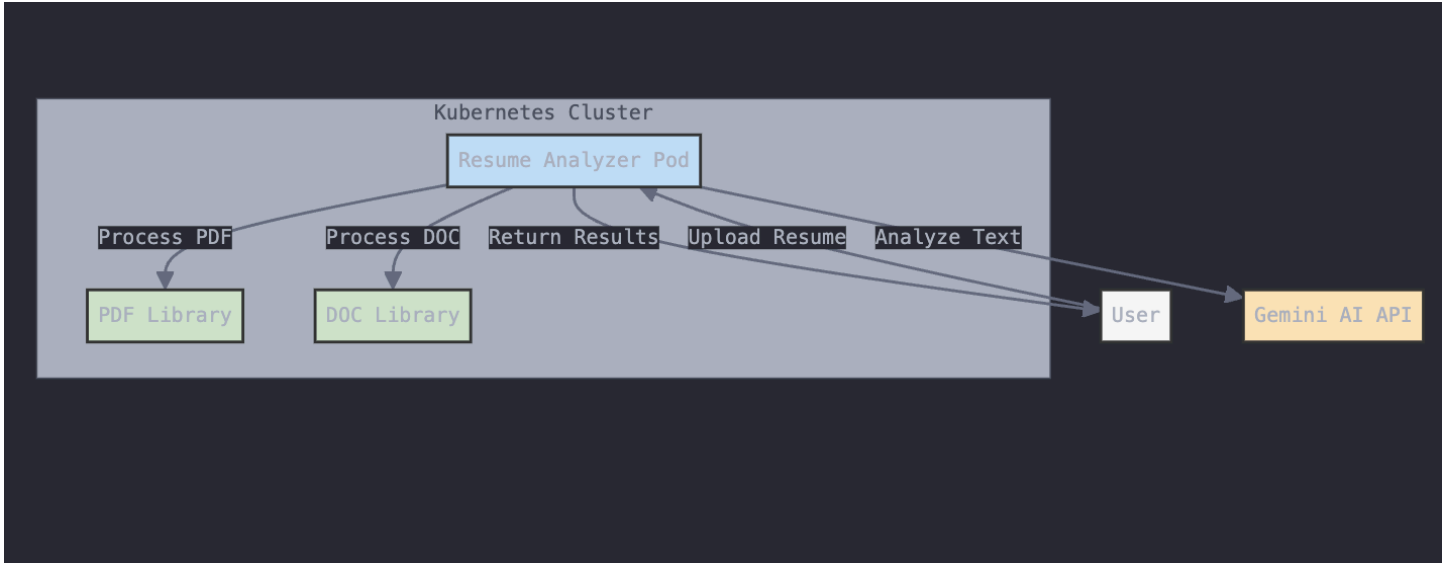
AI-Enhanced Resume Builder

Choose File

No file chosen

Analyze Resume

Architecture diagram



Microservices Architecture

1. Set up Minikube:

```
minikube start --vm-driver=docker
```

2. Enable the Minikube Docker daemon:

```
eval $(minikube docker-env)
```

3. Build Docker images for each service:

```
docker build -t frontend-service ./frontend-service
```

```
docker build -t pdf-service ./pdf-service
```

```
docker build -t doc-service ./doc-service
```

4. Create Kubernetes YAML files (resume-analyzer-secret.yaml and resume-analyzer.yaml).

5. Apply Kubernetes configurations:

```
kubectl apply -f resume-analyzer-secret.yaml
```

```
kubectl apply -f resume-analyzer.yaml
```

6. Verify deployments:

```
kubectl get pods
```

```
(dev) → Resume_Builder git:(main) x kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
doc-service-6b8666c447-6xnq4	1/1	Running	0	64m
frontend-service-5dcbfc9bc7-7zsc4	1/1	Running	0	64m
pdf-service-86cbfcb978-qsk8z	1/1	Running	0	64m

```
(dev) → Resume_Builder git:(main) x
```

7. kubectl get services

```
(dev) → Resume_Builder git:(main) x kubectl get service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
doc-service	ClusterIP	10.108.107.127	<none>	80/TCP	65m
frontend-service	ClusterIP	10.110.237.235	<none>	80/TCP	65m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	68m
pdf-service	ClusterIP	10.106.218.243	<none>	80/TCP	65m

```
(dev) → Resume_Builder git:(main) x
```

8. Access the application:

```
minikube service frontend-service
```

9. View all resources

```
kubectl get all
```

```
(dev) → Resume_Builder git:(main) x kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/doc-service-6b8666c447-6xnq4	1/1	Running	0	62m
pod/frontend-service-5dcbfc9bc7-7zsc4	1/1	Running	0	62m
pod/pdf-service-86cbfcb978-qsk8z	1/1	Running	0	62m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/doc-service	ClusterIP	10.108.107.127	<none>	80/TCP	62m
service/frontend-service	ClusterIP	10.110.237.235	<none>	80/TCP	62m
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	65m
service/pdf-service	ClusterIP	10.106.218.243	<none>	80/TCP	62m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/doc-service	1/1	1	1	62m
deployment.apps/frontend-service	1/1	1	1	62m
deployment.apps/pdf-service	1/1	1	1	62m

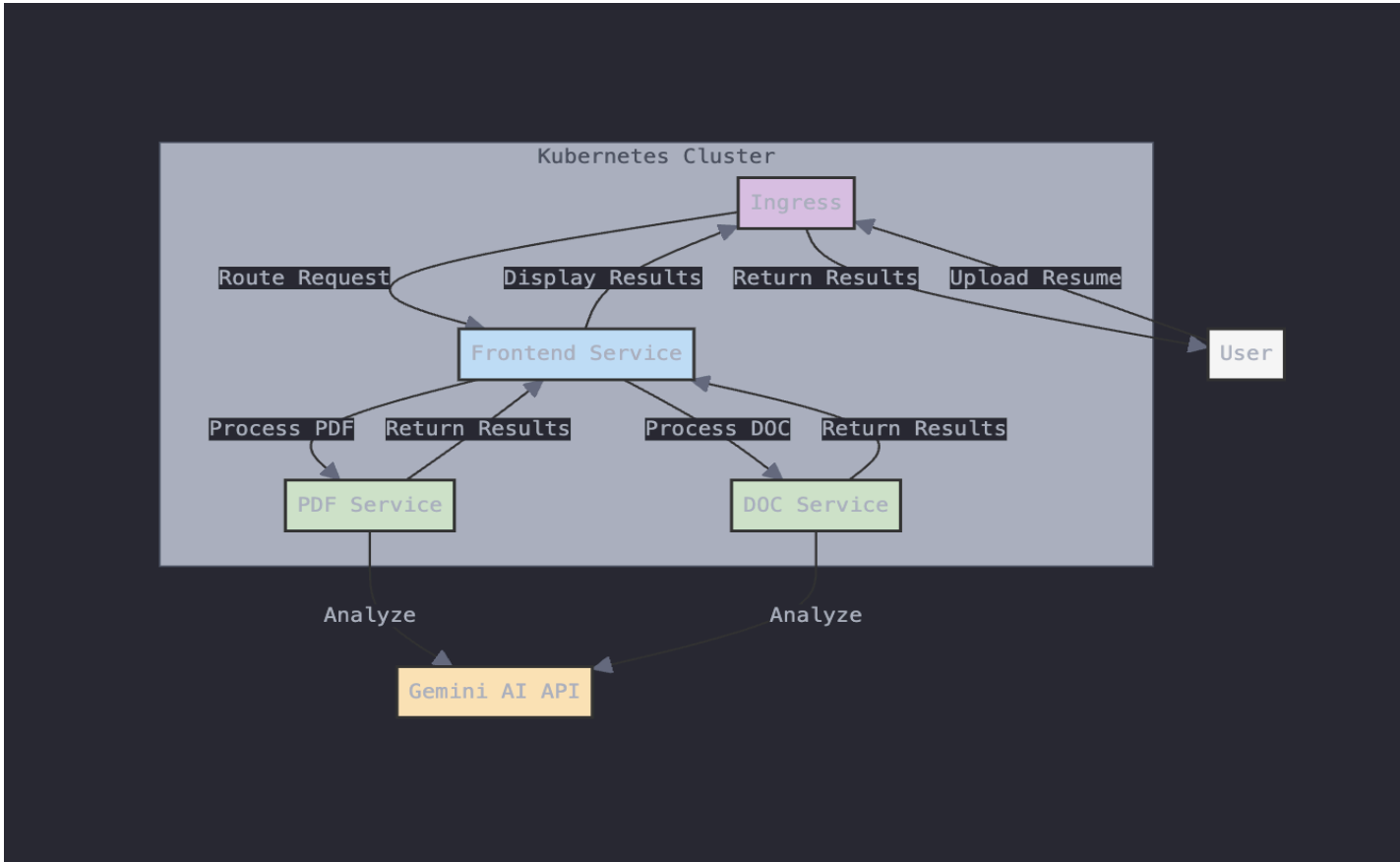
AI-Enhanced Resume Builder

Choose File

No file chosen

Analyze Resume

Architecture Diagram



Minikube dashboard

Workloads

Workloads

Cron Jobs

Daemon Sets

Deployments

Jobs

Pods

Replica Sets

Replication Controllers

Stateful Sets

Service

Ingresses

Ingress Classes

Services

Config and Storage

Config Maps

Persistent Volume Claims

Secrets

Storage Classes

Deployments

Name	Images	Labels	Pods	Created
doc-service	doc-service:latest	-	1 / 1	an hour ago
frontend-service	frontend-service:latest	-	1 / 1	an hour ago
pdf-service	pdf-service:latest	-	1 / 1	an hour ago

Pods

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created
doc-service-6b8666c447-6xnq4	doc-service:latest	app: doc-service pod-template-hash: 6b8666c447	minikube	Running	0	-	-	an hour ago
frontend-service-5dc9bc7-7zsc4	frontend-service:latest	app: frontend-service pod-template-hash: 5dc9bc7	minikube	Running	0	-	-	an hour ago
pdf-service-86cbfc978-qsk8z	pdf-service:latest	app: pdf-service pod-template-hash: 86cbfc978	minikube	Running	0	-	-	an hour ago

Services explanation

1. Frontend Service

The Frontend Service is the primary interface between the user and the application. Its responsibilities include:

- Presenting the user interface for resume upload
- Handling initial file reception from the user
- Determining the file type (PDF or DOC/DOCX) and routing to the appropriate service
- Aggregating results from the processing services and presenting them to the user

Technologies:

- Flask web framework
- HTML/CSS for the user interface

2. PDF Service

The PDF Service is specialized for handling PDF format resumes. Its main functions are:

- Receiving PDF files from the Frontend Service
- Extracting text content from PDF files
- Sending the extracted text to the Gemini AI API for analysis
- Returning the analysis results to the Frontend Service

Technologies:

- Flask for the API endpoints
- PyPDF2 for PDF processing

- Requests library for communicating with Gemini AI API

3. DOC Service

Similar to the PDF Service, the DOC Service specializes in handling DOC and DOCX format resumes. Its responsibilities include:

- Receiving DOC/DOCX files from the Frontend Service
- Extracting text content from DOC/DOCX files
- Sending the extracted text to the Gemini AI API for analysis
- Returning the analysis results to the Frontend Service

Technologies:

- Flask for the API endpoints
- python-docx for DOC/DOCX processing
- Requests library for communicating with Gemini AI API

4. Ingress

While not a microservice itself, the Ingress is a crucial component in our microservices architecture:

- Acts as the entry point for external traffic into the Kubernetes cluster
- Routes incoming requests to the appropriate service based on the URL path
- Provides a single point of access for the application, simplifying the external interface

Technologies:

- Kubernetes Ingress resource
- Typically implemented using NGINX Ingress Controller in many Kubernetes setups

Interactions Between Services

1. The user interacts with the Frontend Service through the Ingress.
2. Based on the file type, the Frontend Service sends the resume to either the PDF Service or the DOC Service.
3. The PDF/DOC Service processes the file, extracts text, and sends it to the Gemini AI API for analysis.
4. The analysis results are sent back through the service chain: PDF/DOC Service → Frontend Service → Ingress → User.